Lab Assignment 2:
System Refactoring Driven by Quality Attributes

DVA506 Software Architecture

Project Group 4

2026-03-17

Umair Khalid
Balkrishna Panthi
Shaik Omar Farooq
Stina Häggmark
Asli Sümen

Event-Driven / Producer-Consumer Pattern

Before refactoring, the simulation classes called the controller directly.

It is refactored to:

- let simulators create events
- send those events to an EventBus
- let an EventProcessor consume the events
- delegate the actual work to dedicated services

This makes the system less tightly coupled and easier to extend.

Main Changes Made

1. The controller is simplified

Originally, the controller handled many responsibilities at the same time, including:

- rental logic
- telemetry logic
- state transitions
- audit logging

It is refactored to become a thin coordination layer.

Two service classes are introduced:

- RentalService
- TelemetryService

These classes now contain the main business logic.

Tactics used:

- Decompose responsibilities
- Separate concerns

```python
class SmartMoveCentralController:
    def __init__(self):
        self.audit = AuditLog()
        self.active_rentals = {}


    def start_rental(self, user, vehicle):
        with vehicle.lock:
            if vehicle.state != VehicleState.AVAILABLE:
                raise Exception("Vehicle not available")

            # City rules before unlock
            rules = CITY_RULES[vehicle.city.value]
            rules.on_unlock(vehicle, user)


            rental = Rental(
                rental_id=str(uuid.uuid4()),
                user_id=user.id,
                vehicle_id=vehicle.id
            )

            self.active_rentals[vehicle.id] = rental
            old_state, new_state = StateMachine.transition(vehicle, VehicleState.IN_USE)

            self.audit.record(
                vehicle.id,
                f"{old_state.name} -> {new_state.name}",
                "Rental started"
            )

            return rental

    def end_rental(self, vehicle):
        with vehicle.lock:
            rental = self.active_rentals.get(vehicle.id)
            if not rental:
                raise Exception("No active rental")

            rental.end()
            rental.cost = PricingEngine.calculate(rental)

            # City-specific billing rules
            rules = CITY_RULES[vehicle.city.value]
            rules.on_end_rental(rental)

            old_state, new_state = StateMachine.transition(vehicle, VehicleState.AVAILABLE)
            del self.active_rentals[vehicle.id]

            self.audit.record(
                vehicle.id,
                f"{old_state.name} -> {new_state.name}",
                f"Rental ended. Cost: {rental.cost}"
            )

            return rental
```

(a) before refactoring

```python
class SmartMoveCentralController:

    def __init__(self, vehicles=None, users=None, audit_log=None):
        self.vehicles = vehicles or {}
        self.users = users or {}
        self.audit = audit_log or AuditLog()
        self.active_rentals = {}

        self.rental_service = RentalService(
            vehicles=self.vehicles,
            users=self.users,
            audit_log=self.audit,
            active_rentals=self.active_rentals
        )

        self.telemetry_service = TelemetryService(
            vehicles=self.vehicles,
            audit_log=self.audit,
            active_rentals=self.active_rentals,
            rental_service=self.rental_service
        )

    def bind_runtime_state(self, vehicles, users):
        """
        Allows simulation/bootstrap to inject loaded state after construction.
        Keeps dependencies explicit and improves testability.
        """
        self.vehicles = vehicles
        self.users = users

        self.rental_service.vehicles = vehicles
        self.rental_service.users = users

        self.telemetry_service.vehicles = vehicles

    def start_rental(self, user, vehicle):
        return self.rental_service.start_rental(user, vehicle)

    def end_rental(self, vehicle):
        return self.rental_service.end_rental(vehicle)
```

(b) after refactoring

Figure 1: Refactoring sample from the SmartMoveCentralController class

2. Event-based communication is introduced

Originally:

- TelemetrySimulator called controller methods directly
- RentalSimulator called controller methods directly

It is refactored to use:

- EventBus
- EventProcessor

Now:

- simulators publish events
- the event bus stores them in a queue
- the event processor consumes the events

- the appropriate service handles the event

Tactics used:

- Introduce an intermediary
- Decouple producers and consumers

This improves maintainability and scalability.

3. Telemetry simulation is made more concurrent

Originally, telemetry was generated by one loop in one thread.

It is refactored to use:

- multiple TelemetryWorker threads
- partitioned subsets of vehicles per worker

This allows multiple groups of vehicles to produce telemetry concurrently.

Tactics used:

- Introduce concurrency
- Partition work

This improves performance and scalability.

4. Audit logging is strengthened

Originally, the audit log recorded events in a simpler form.

It is refactored to include:

- thread-safe writes using a lock
- sequential log entry IDs
- checksum chaining
- integrity verification

Tactics used:

- Add synchronization
- Add integrity checking
- Add redundancy for verification

This improves reliability and integrity.

5. File storage is made safer

Originally, persistence relied on simpler file writes.

It is refactored to use atomic JSON writes, so files are replaced safely instead of being overwritten unsafely.

Tactics used:

- Use atomic operations
- Improve fault tolerance

This improves reliability.

6. Business logic is made easier to test

Originally, logic was spread across controller and simulator code.

It is refactored to move core logic into smaller service classes and event handlers.

A test file is also added for:

- state transitions
- rental flow
- audit integrity

Tactics used:

- Isolate responsibilities
- Create testable boundaries

This improves testability.

Quality Attributes Improved

1. Maintainability

It is improved by:

- simplifying the controller
- moving logic into dedicated services
- introducing event-based communication
- reducing direct dependencies between components

Tactics used:

- Decompose responsibilities
- Separate concerns
- Introduce intermediary layer

Applied in:

- core/controller.py
- core/services/rental_service.py
- core/services/telemetry_service.py
- core/event_bus.py
- core/event_processor.py

```python
def process_telemetry_event(self, vehicle, event):
  with vehicle.lock:

      current_state = vehicle.state

      # Do nothing if already in maintenance
      if current_state == VehicleState.MAINTENANCE:
          return

      rules = CITY_RULES[vehicle.city.value]
      # ----------------------------------------------------
      # Overheating
      # ----------------------------------------------------
      if event.temperature > 60 and current_state == VehicleState.IN_USE:

          old, new = StateMachine.transition(
              vehicle,
              VehicleState.EMERGENCY_LOCK
          )

          self.audit.record(
              vehicle.id,
              f"{old.name} -> {new.name}",
              "Emergency lock: Overheating detected"
          )

          rental = self.active_rentals.get(vehicle.id)
          if rental:
              rental.end()
              del self.active_rentals[vehicle.id]

          return
      # ----------------------------------------------------
      # Critical low battery
      # ----------------------------------------------------
      if event.battery < 5:

          if current_state == VehicleState.IN_USE:
              rental = self.active_rentals.get(vehicle.id)
              if rental:
                  rental.end()
                  del self.active_rentals[vehicle.id]

          if StateMachine.validate(vehicle.state, VehicleState.MAINTENANCE):

              old, new = StateMachine.transition(
                  vehicle,
                  VehicleState.MAINTENANCE
              )

              self.audit.record(
                  vehicle.id,
                  f"{old.name} -> {new.name}",
                  "Maintenance required: Battery critically low"
              )

          return
```

(a) before refactoring

```python
def _handle_overheating(self, vehicle, event):
    # Overheating
    if event.temperature > 60 and vehicle.state == VehicleState.IN_USE:
        old_state, new_state = StateMachine.transition(
            vehicle,
            VehicleState.EMERGENCY_LOCK
        )

        self.audit.record(
            entity_id=vehicle.id,
            action=f"{old_state.name} -> {new_state.name}",
            reason="Emergency lock: Overheating detected"
        )

        self.rental_service.terminate_active_rental(
            vehicle,
            "Rental terminated due to overheating"
        )
def _handle_critical_battery(self, vehicle, event):
    # Critical battery
    if event.battery < 5:
        if vehicle.state == VehicleState.IN_USE:
            self.rental_service.terminate_active_rental(
                vehicle,
                "Rental terminated due to critically low battery"
            )

        if StateMachine.validate(vehicle.state, VehicleState.MAINTENANCE):
            old_state, new_state = StateMachine.transition(
                vehicle,
                VehicleState.MAINTENANCE
            )

            self.audit.record(
                entity_id=vehicle.id,
                action=f"{old_state.name} -> {new_state.name}",
                reason="Maintenance required: Battery critically low"
            )

def process_telemetry_event(self, vehicle, event):
    with vehicle.lock:
        self._sync_telemetry(vehicle, event)

        # maintenance vehicles remain isolated from regular telemetry flow
        if vehicle.state == VehicleState.MAINTENANCE:
            return

        if self._handle_emergency_lock(vehicle, event):
            return

        if self._handle_unauthorized_movement(vehicle, event):
            return

        if self._handle_city_restrictions(vehicle, event):
            return

        if self._handle_overheating(vehicle, event):
            return

        self._handle_critical_battery(vehicle, event)
```

(b) after refactoring

Figure 2: Refactoring sample from the process_telemetry_event() function

2. Reliability

It is improved by:

- thread-safe audit writes
- controlled state transitions through StateMachine
- atomic persistence writes
- clearer exception structure

Tactics used:

- Synchronization

- Validate before change
- Atomic write
- Controlled error handling

Applied in:

- persistence/audit_log.py
- core/state_machine.py
- core/exceptions.py
- persistence/storage.py

3. Performance / Scalability

It is improved by:

- multiple telemetry workers
- event queue between producers and consumers
- partitioning telemetry work across workers

Tactics used:

- Introduce concurrency
- Partition work
- Decouple processing

Applied in:

- simulation/telemetry_worker.py
- simulation/telemetry_simulator.py
- simulation/simulation_engine.py
- core/event_bus.py
- core/event_processor.py

4. Integrity / Security

It is improved by:

- checksum chaining in audit log
- sequential IDs in audit entries
- integrity verification support

Tactics used:

- Integrity verification
- Checksum chaining
- Sequential logging

Applied in:

- persistence/audit_log.py

5. Testability

It is improved by:

- increase code coverage by adding unit tests that are implemented for critical paths, reaching a project-wide coverage of 81.3%.
- edge case verification by adding some specific tests for city-specific exceptions (e.g., RestrictedZoneError for Rome and HelmetRequiredError for Milan).
- regression safety with a dedicated test suite was created for the TelemetryProcessor to ensure that battery and temperature thresholds always trigger the correct safety transitions (EMERGENCY_LOCK or MAINTENANCE).
- extracting logic into smaller services
- separating simulation from business logic

Tactics Used:

- Isolate Core Logic: Business rules (Milan helmet checks, London charges, Rome restricted zones) were moved from the Controller into specialized RentalService and TelemetryService classes.
- Encapsulation of state transitions
- Use of mocking and interfacing by MagicMock
- Add testable boundaries

Applied in:

- smartmove/core/services/rental_service.py (Isolated rental logic)
- smartmove/core/services/telemetry_service.py (Isolated rule validation)
- smartmove/core/telemetry.py (Refactored for 100% test coverage)
- tests/test_city_rules.py (Validation of city-specific business constraints)
- tests/test_telemetry.py (Safety threshold and transition verification)
- tests/test_main.py (Testing the entry point using mock-breaking of infinite loops)

Summary

The SmartMove system is refactored to improve:

- maintainability
- reliability
- performance/scalability
- integrity
- testability

The main architectural pattern applied is:

- Event-Driven / Producer-Consumer

The main tactics applied include:

- decomposition
- separation of concerns
- synchronization
- atomic operations
- integrity verification
- concurrency
- work partitioning
- intermediary/event queue introduction

The refactored version is more modular, more robust, and better aligned with the assignment requirements.